Normalización

Entrenamiento de redes neuronales

Normalizar datos

- La normalización es una **técnica de preparación de datos.**
- La normalización es un proceso que permite transformar las variables de un rango de valores a una escala numérica común.
- Es decir que al normalizar se **ajustan los valores de entrada (algunas veces de salida) para que estén dentro de un rango mas pequeño** y consistente:
  - Valores entre **0 y 1** o bien entre **-1 y 1**.

Ejemplo:

- Sin normalizar: **0, 10, 20, 30, 40**
- Normalizados [0-1]: **0.0, 0.25, 0.50, 0.75, 1.0**

Porque es necesario normalizar?

- Desea implementar un programa que prediga el precio de un vehículo usados utilizando dos datos de entrada:
  - **Antigüedad:** de 0 a 20 años.
  - **Precio original:** de 10,000 a 100,000 USD.

- Cuando la red neuronal analiza estos datos, el número "**100,000**" es **5,000** veces más grande que "**20**". La red neuronal pensará matemáticamente que el precio es muchísimo más importante que la antigüedad, simplemente porque el número es mayor.

  - **La normalización pone a ambos en el mismo nivel de importancia.**

Beneficios de normalizar en ML

- Velocidad de Entrenamiento
- Reducción en la perdida
- Facilidad en la convergencia del modelo
- Estabilidad Numérica, previniendo errores tipo **Nan (not a number)** o **inf (∞)** o el colapso del modelo
- Justicia entre Variables al asegurar que todas las variables contribuyan al entrenamiento equitativamente

Métodos mas comunes

Método	Fórmula	Resultado	Cuándo usarlo
Escalado Min-Max	$\frac{x-min}{max-min}$	Datos entre **0** y **1**	Cuando conoce los límites (ej. píxeles de imagen).
Estandarización	$\frac{x-\mu}{\sigma}$	Media = **0**, Desv. Est. = **1**	Cuando los datos tienen valores atípicos (*outliers*).

Codigo en Pythom

```
#No normaliza pero prepara la herramienta
scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()

#Normaliza: encuentra los maximos y minimos, posterirmente aplica la formula de escalado en normalizacion
X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y)

#Desnormalizar
y_scaled = scaler_y.inverse_transform(y)
```

Demostración

Conclusión

Normalizar:

- No cambia la información esencial de los datos.
- Mejora la estabilidad numérica.
- Acelera la convergencia.
- Facilita que la red encuentre una solución óptima.

Ejercicio

Predicción de precios de casas
En este ejercicio intentará construir una red neuronal que prediga el precio de una casa de acuerdo con una fórmula simple.

Si el precio de la casa fuera tan fácil como que una casa cuesta 50000 + 50000 por habitación, de modo que una casa de 1 habitación cuesta 100000, una casa de 2 habitaciones cuesta 150000, etc.

¿Cómo crearía una red neuronal que aprenda esta relación para predecir que una casa de 7 habitaciones cuesta cerca de 400000, etc.?

Ejercicio

Una planta industrial desea predecir el consumo de energía diaria en Kilovatios-hora (kWh) en función de la producción diaria en toneladas procesadas. Los datos recolectados durante 12 días de operación son los siguientes:

Dia	Produccion (X) [Toneladas]	Consumo (Y) [kWh]
1	10	1,200
2	20	1,850
3	30	2,400
4	40	3,100
5	50	3,750
6	60	4,300
7	70	5,000
8	80	5,600
9	90	6,250
10	100	6,900
11	110	7,500
12	120	8,200

Ejercicios

Ejercicio 1: predecir el puntaje de un examen según horas de estudio y asistencia a clases (Sí / No).

Horas de estudio	Asistencia	Puntaje
2	Sí	70
1	No	50
3	Sí	85
0	No	40

Ejercicio 2: predecir la temperatura final del café después de 10 minutos según la temperatura inicial (°C) y el tipo de taza (Plástico, Cerámica, Vidrio).

Temp inicial	Tipo de taza	Temp después de 10 min (centígrados)
85	Plástico	75
90	Cerámica	80
95	Vidrio	78
88	Plástico	78

Obtener: Aplicar one-hot encoding a la columna alfa-numerica, Normalizar entradas y salida, crear un programa interactivo que pregunte la temperatura inicial y tipo de taza y prediga la temperatura final.